Playing Gridshot with an SO-101 Robotic Arm

Harsha Addagada Joshua Cabebe Natiq Ghafoor Varun Mange

***Abstract*— We investigate whether an SO-101 robotic arm, fitted with a custom mouse-gripping attachment, can physically play Gridshot by translating visual targets into real-time mouse movements and clicks. Our system follows a perception-planning-control pipeline: a webcam identifies target and cursor positions on screen, a motion planner converts these into physical mouse movements, and the arm executes the resulting trajectory. We evaluate the system using Gridshot's built-in metrics (score, accuracy, per-hit latency).**

I. INTRODUCTION

Gridshot is an aim-training task in which a player moves a cursor and clicks targets as quickly and accurately as possible. In the AimCrane version, three targets appear at a time, and each successful hit causes a new target to appear elsewhere on the grid. Our project investigates whether an SO-101 robotic arm can perform this task by physically moving and clicking a computer mouse. Our goal is to develop a working system and compare its performance with our team members under the same game settings.

We chose this project because it combines perception, motion planning, and manipulation in a task with clear performance measures. Although using a mouse is familiar to humans, performing the same action with a robotic arm requires coordination between perception and movement. The robot must identify a target, determine the required mouse movement, and click while keeping the cursor aligned. Moving faster may reduce the time between hits, but can also increase positioning errors or cause the mouse to slip.

Our proposed system follows a perception, planning, and control pipeline. A webcam will capture the game screen to identify visible targets and the cursor position. An overhead camera will track ArUco markers placed around the mousepad and on the mouse to determine its physical position and orientation. The system will select the target closest to the cursor and use a calibrated relationship between physical mouse movement and cursor displacement to determine the required motion. The arm will then move the mouse toward the target and click once the cursor is sufficiently aligned.

We have developed an initial script for detecting circular targets and designed and 3D-printed a custom mouse attachment. The detection script identifies target locations, while the attachment helps the arm hold and move the mouse consistently. The next stage is to calibrate the relationship between physical mouse movement and cursor displacement and connect the detection script to the arm's motion control. Once integrated, we will evaluate the system using game score, accuracy, successful hits, misses, and average time per hit and compare its performance with our team members.

II. RELATED WORK

Previous research has explored robotic manipulation of physical computer mice. He et al. [1] developed a soft robotic hand mounted on a *UR5e* robotic arm capable of performing 11 mouse operations, including clicking, scrolling, and grasping. Their custom end effector combines soft pneumatic actuators with four servos to provide the dexterity required for mouse manipulation. They also demonstrated an automated target-shooting application using computer vision to detect on-screen targets and cursor positions. By calibrating the relationship between physical mouse movement and cursor displacement, their system could move the mouse toward targets and click when aligned. This work closely resembles our proposed perception, planning, and control pipeline but uses a more complex end effector and an industrial robotic arm.

Robotic systems have also been developed specifically for aim-training games. Carter [2] demonstrated a physical aimbot using an omnidirectional wheel-based mechanism to control a mouse and play Aim Lab. Another project, irlAimBot [3], used linear actuators and OpenCV-based target detection to physically aim and click targets in Gridshot.

For robotic motion planning, PyRoki is a modular toolkit for robotic kinematic optimization [4]. It supports forward kinematics (FK) and inverse kinematics (IK), allowing us to calculate the SO-101's end-effector pose from joint angles and determine the joint configurations needed to reach a desired mouse position. Additionally, Ruckig is a real-time trajectory generation algorithm that produces time-optimal, jerk-limited trajectories while respecting joint velocity, acceleration, and jerk constraints [5]. These tools provide a foundation for generating smooth, controlled mouse movements while respecting the SO-101's joint limits.

While these projects demonstrate robotic mouse manipulation and automated aiming, we found no prior work using the SO-101 robotic arm for this task. Our project explores this application using the SO-101 with custom mouse-holding and clicking attachments, integrating computer vision, motion planning, and physical mouse control.

Mouse Holder Design

The SO-101's standard gripper has two fingers, only one of which is actuated. To reliably grasp and move the mouse, we considered adding more fingers or designing a custom mouse holder. We chose to design a mouse-specific holder that attaches directly to the SO-101's wrist-roll joint. We developed three iterations of the mouse holder. The first two had loose tolerances, causing the mouse to slip during movement. For the third iteration, we revised our design



Fig. 1. Mouse holder for SO-101 (Left - CAD design; Right - 3D printed holder attached to SO-101)

approach to achieve a tighter fit (Fig. 1), allowing the holder to securely grip the mouse even during rapid, sharp movements.

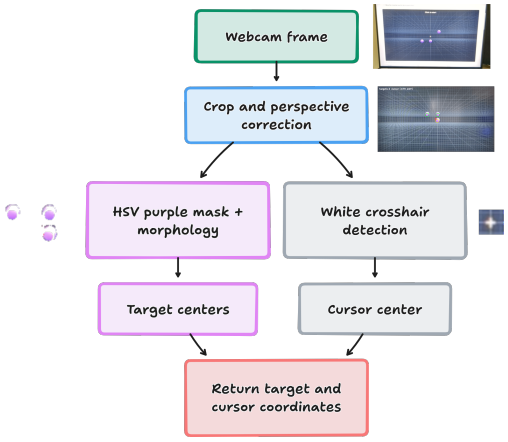
We also designed a custom finger attachment to press the left mouse button. The original gripper finger restricted the range of motion needed for clicking, so we replaced it with a curved, L-shaped finger. This design allows the SO-101 to reliably press the left mouse button, enabling it to shoot targets in Gridshot.

III. METHOD

A. *Perception*

Our perception system consists of two components: target and cursor detection on the game screen and physical mouse tracking on the mousepad. Both will use external cameras and OpenCV to provide inputs for motion planning and control.

1) Target and Cursor Detection: We will use a computer vision pipeline to detect Gridshot targets and the cursor using an external camera. Currently, we plan on using an external camera instead of doing screen capture because we want to avoid running any software on the computer running the Gridshot game (we will perform experiments to check the performance of the external camera and screen capture setups).



The pipeline will automatically identify the game region using contour detection and apply a perspective transformation to obtain a rectangular cropped game view. Since Gridshot targets are bright purple circles (Fig. 2), we will use **HSV** color segmentation followed by morphological operations to isolate them. Local peak detection will be used to identify target centers.

For cursor detection, we will use the crosshair's distinctive white, plus-shaped feature. We will construct a whiteness map using pixel brightness and saturation, then compare pixel intensities along the horizontal, vertical, and diagonal directions to distinguish the crosshair from other bright objects. The resulting target and cursor coordinates will be passed to the motion planner.

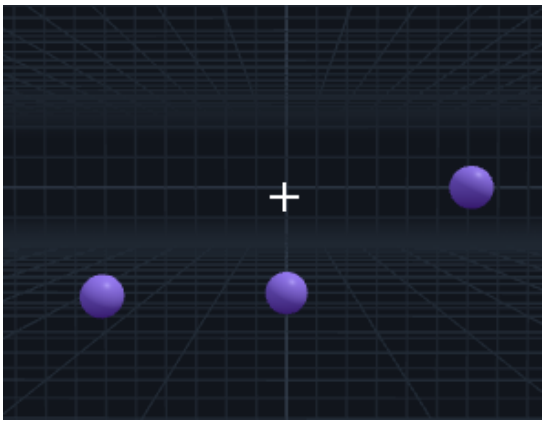


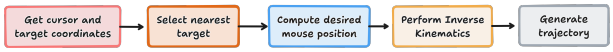
Fig. 2. Sample Gridshot game scene

2) Mouse Tracking: We will have an overhead camera-based tracking system to estimate the mouse's physical position and orientation on the mousepad. This information will help calibrate physical mouse movement against cursor displacement, verify commanded movements, and ensure the mouse stays within the workspace.

The system will use four 25 mm ArUco markers (IDs 0-3) at the workspace corners and a fifth marker (ID 9) attached to the mouse (Fig. 3). A webcam will stream 1920 x 1080 video, which is processed using OpenCV [6]. At startup, each corner marker's detected corners will be averaged over 15 frames. Since every marker is a square of known size lying in the plane of the pad, one Levenberg-Marquardt fit to the 16 marker corners estimates both the plane-to-image homography [7] and the position and rotation of markers 1-3 relative to marker 0. The printed marker size sets the scale, so the distances between the markers never need to be measured by hand. The workspace frame has its origin at marker 0, with $+x$ toward marker 1 and $+y$ toward marker 2.

For each subsequent frame, the mouse marker is transformed into workspace coordinates. Its center determines the mouse position, while its top edge determines orientation. Consecutive observations allow us to estimate movement and check workspace boundaries. We will also monitor calibration-marker drift and apparent marker scale to detect camera movement and potential perspective errors.

B. *Planning*



1) Mouse Calibration: We will first calibrate the relationship between physical mouse displacement (mm) and cursor displacement (pixels). With fixed game resolution, mouse DPI, and sensitivity, we can estimate this mapping by moving the mouse known distances in different directions and measuring the resulting cursor movement. Using the calibrated mapping, we can determine the physical mouse movement required to move the cursor from its current position to the selected target.

2) Forward and Inverse Kinematics: We will use the SO-101's kinematic model to relate joint angles to the mouse's physical position and orientation. Forward kinematics will estimate the current mouse pose, while inverse kinematics (IK) will compute joint configurations for desired mouse positions. We plan to use an existing kinematics library, such as PyRoki, rather than implement an IK solver from scratch [4].

To maintain contact with the mousepad, we will constrain the mouse to a fixed height (z) and orientation, allowing movement only along the mousepad's horizontal plane (x and y axes). The planner will also enforce joint limits and workspace boundaries to avoid unreachable configurations.

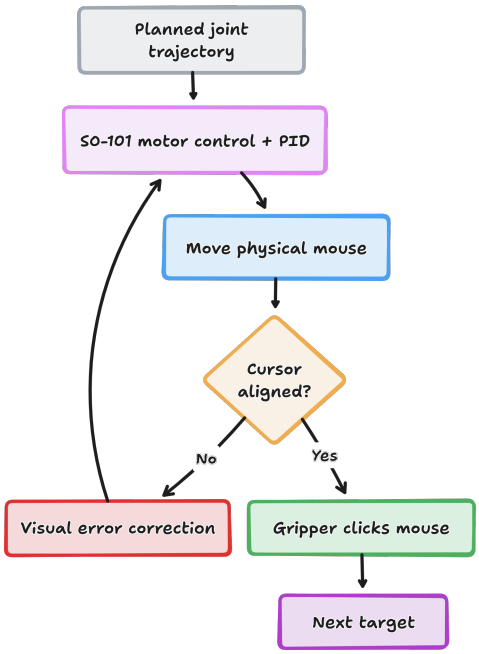
3) Trajectory Generation: Once the desired mouse position is determined, we will generate a smooth trajectory from the current position to the target position. The trajectory will be divided into intermediate waypoints, each maintaining the same height and orientation. Inverse kinematics will then compute the corresponding joint angles at each waypoint.

We plan on using the Ruckig library to generate smooth, time-parameterized joint trajectories while respecting joint velocity, acceleration, and jerk limits [5]. The resulting joint-space trajectory will be passed to the controller for execution. Visual feedback from the game screen will allow the system to correct any remaining cursor-position error before clicking.

C. *Control*

The control system will execute the planned joint trajectories while using feedback from the robotic arm and game screen to improve positioning accuracy. We plan to use a feedforward and feedback control approach to track the joint angles generated by the motion planner. The feedforward component will provide the desired joint angle at each time step, while feedback from the SO-101's joint encoders will be used to measure tracking errors. We will initially rely on the servos' built-in position controllers and explore additional PID-based correction if necessary.

Errors caused by mouse friction, sensor inaccuracies, and imperfect calibration may result in the cursor not reaching the target. To compensate, we will implement a visual feedback system that continuously compares the detected cursor position with the target position. If the positioning error exceeds a defined tolerance, the system will calculate the remaining displacement and command a corrective mouse movement.



Once the cursor is sufficiently inside the target area, the controller will actuate the SO-101's gripper motor through our custom clicking attachment to press and release the left mouse button. The system will then detect the next target and repeat the process.

IV. EXPERIMENTS

A. *Overhead Mouse Tracking*

We tested the overhead tracker with marker 9 printed on paper and slid by hand around a 252×212 mm workspace for 26 s (Fig. 3). The arm and mouse were not yet involved. Table I summarizes the results, taken from the tracker's on-screen readout and its saved calibration files.

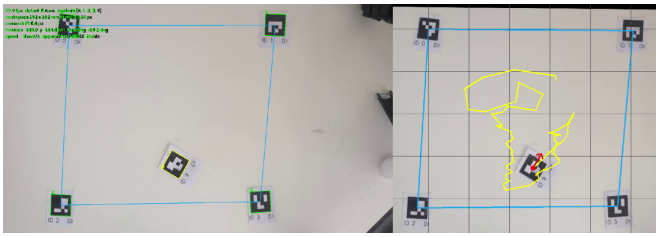


Fig. 3. Overhead mouse tracking. Left: camera view with the four corner markers (IDs 0-3, green), the fitted workspace outline (blue), and the mouse marker (ID 9, yellow). Right: rectified top-down view of the pad with a 50 mm grid, the marker's trail over the 26 s test (yellow), and its current position and heading (red).

The calibration was accurate and repeatable. Over three calibrations, the RMS reprojection error of the fit was 0.30–0.56 px, or about 0.08–0.14 mm on the pad at the camera's scale of roughly 0.25 mm per pixel. Two calibrations of the same marker layout taken 37 s apart placed every corner

TABLE I

OVERHEAD MOUSE TRACKING TEST

Quantity	Value
Workspace	252 × 212 mm
Markers	ArUco 4 × 4, 25 mm
Camera stream	1920 × 1080, 16–32 fps
Marker detection time	6–9 ms per frame
Calibration RMS error	0.30–0.56 px (≈0.08–0.14 mm)
Repeat calibration	corners agree within ≈0.1 mm
Corner drift during run	0.3–0.7 px
Mouse marker scale check	within 1% of 25 mm

marker within about 0.1 mm of each other. During the run the corner markers stayed within 0.3–0.7 px of their calibrated positions, so the camera did not move. Marker 9, which the calibration does not use, measured within about 1% of its printed 25 mm size across the workspace, an independent check of the scale.

For comparison, at an example mouse resolution of 800 DPI with the default game sensitivity, a medium target's hit radius corresponds to about 1.6 mm of mouse travel and one grid column to 13.8 mm. The calibration error is about ten times smaller than the hit tolerance. We have not yet measured the frame-to-frame jitter of a moving marker, so this is not yet a complete accuracy figure.

Marker 9 was occasionally lost while being moved by hand when the fingertip partly covered it (in 2 of 9 readouts sampled every 3 s). On the robot, the gripper and mouse attachment must leave the marker in clear view.

B. Target and Cursor Detection

We tested our target and cursor detection script on a recorded Gridshot session to evaluate its accuracy and real-time performance. The script processes each frame to identify the three purple targets and the cursor position (Fig. 4). The targets are circled in green and yellow. Yellow signifies the closest target to the cursor. The cursor is marked with a red + symbol. We evaluated the system using four metrics: processing frame rate, perception latency, number of detected targets, and detection confidence.

Fig. 5 shows the results from a 41-second test using a 30 FPS video source. The processing loop maintained approximately 16-21 FPS, below the source frame rate. Perception latency ranged from roughly 30–50 ms, frequently exceeding the 33.3 ms processing budget required for 30 FPS operation. This indicates that the current detection pipeline cannot consistently process every incoming frame in real time.

Despite these performance limitations, the detector consistently identified the three visible targets throughout the experiment. Occasional detections of four targets suggest false positives, while the detection confidence (detected purple blob's radius divided by the maximum expected target radius) generally remained between 0.4 and 0.6, with some fluctuations.

Overall, the experiment demonstrates that our perception system can reliably identify targets and track the cursor

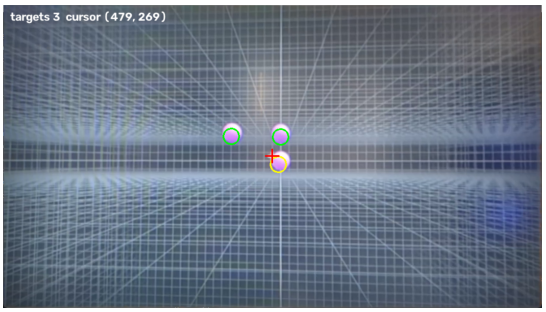


Fig. 4. Target and cursor detection
Targets are circled in green, the closest target is circled in yellow, and the red plus (+) is the cursor

position. Further optimization is needed to reduce processing latency, but our system does not necessarily need to process every frame. We will perform further experiments to decide on the best frame rate for our target and cursor detection pipeline.

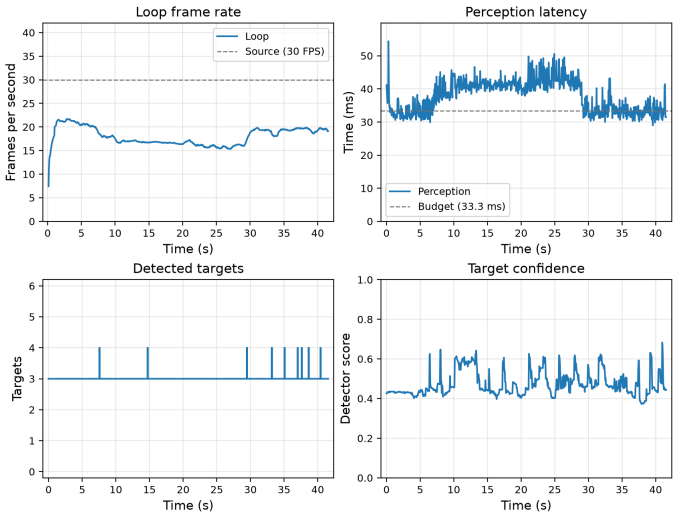


Fig. 5. Target and cursor detection metrics

V. NEXT STEPS

We will focus on integrating the perception, planning, and control components into a complete system.

1) **Mouse-to-Cursor Calibration** - we will establish the relationship between physical mouse movements and cursor displacement to accurately translate screen coordinates into mouse movements.
2) **Improve Target and Cursor Detection** - we will polish the detection script to reliably identify targets and the cursor under varying lighting conditions, screen configurations, and other edge cases.
3) **Adapt ArUco Marker Tracking** - we will integrate the existing ArUco-based tracking system with our mouse and mousepad setup to accurately track the mouse's position and orientation.
4) **Motion Planning and Control** - we will develop and implement motion-planning and control algorithms

to move the SO-101 toward selected targets while respecting joint limits, maintaining mousepad contact, and correcting positioning errors using visual feedback. We will mostly use pre-existing libraries for motion planning.

5) **Performance Evaluation** - we will repeatedly test our robot and program in Gridshot and compare the robot's performance against our team members using game score, successful hits, misses, accuracy, and per-hit latency. Our aim is to reach a game score greater than 5000.

REFERENCES

[1] Y. He, D. Yan, K. Jin, B. Fang, and S. Li, "A robotic hand with soft transmission systems for automated operations of computer mouse," *Advanced Intelligent Systems*, vol. 7, no. 8, p. 2400966, 2025. [Online]. Available: https://advanced.onlinelibrary.wiley.com/doi/abs/10.1002/aisy.202400966

[2] K. Carter, "Robot that makes me an Aiming Pro | Physical Aimbot." [Online]. Available: https://www.youtube.com/watch?v=ne9bmMX82iY

[3] B. Drobnak, "bdrobn/irlAimBot," Jun. 2026, original-date: 2024-09-28T15:44:44Z. [Online]. Available: https://github.com/bdrobn/irlAimBot

[4] C. M. Kim*, B. Yi*, H. Choi, Y. Ma, K. Goldberg, and A. Kanazawa, "Pyroki: A modular toolkit for robot kinematic optimization," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025. [Online]. Available: https://arxiv.org/abs/2505.03728

[5] L. Berscheid and T. Kröger, "Jerk-limited real-time trajectory generation with arbitrary target states," *Robotics: Science and Systems XVII*, 2021. [Online]. Available: https://arxiv.org/abs/2105.04830

[6] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[7] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.